📚 FastAPI Complete Syllabus — Beginner to Advanced

> A structured course from zero to production-grade FastAPI. Each topic gets its own `Lesson N - <Topic>` folder with `theory.md` + `main.py`.

🟢 PHASE 1 — FUNDAMENTALS (Beginner)

- [x] **1. What is FastAPI**
- [x] **2. Installation & Project Setup**
  - Virtual environments (`venv`, `uv`, `poetry`)
  - Installing `fastapi`, `uvicorn`
  - Project folder basics
- [x] **3. First API — GET & POST**
  - Decorators (`@app.get`, `@app.post`, `@app.put`, `@app.delete`)
  - HTTP methods explained
- [x] **4. Path Parameters**
  - Type conversion
  - Path parameter validation (`Path()`)
  - Order of routes (why it matters)
- [x] **5. Query Parameters**
  - Optional vs required
  - Default values
  - `Query()` validation (min_length, regex, etc.)
- [x] **6. Request Body with Pydantic Models**
  - `BaseModel`
  - Field types
  - Nested models
  - `Field()` validation
- [x] **7. Combining Path + Query + Body**
- [x] **8. Response Basics**
  - Auto JSON conversion
  - Status codes (`status_code=`)
  - `JSONResponse` vs returning a dict

🟡 PHASE 2 — CORE FEATURES (Intermediate)

- [x] **9. Pydantic Deep Dive**
  - Validators (`@field_validator`, `@model_validator`)
  - Config (`model_config`)
  - `Field` constraints
  - Pydantic v1 vs v2 differences
- [x] **10. Response Models**
  - `response_model=`
  - Filtering output fields
  - `response_model_exclude_unset`, `exclude_none`
- [x] **11. Multiple Models per Route**
  - Input model vs Output model
  - `UserCreate`, `UserResponse`, `UserUpdate` pattern
- [x] **12. Status Codes & HTTP Exceptions**
  - `HTTPException`
  - Custom exception handlers
  - `status` module
- [x] **13. Error Handling**
  - Validation errors
  - Custom error responses
  - Global exception handlers
- [x] **14. Dependency Injection (huge topic)**
  - `Depends()`
  - Sub-dependencies
  - Class-based dependencies
  - Dependency caching
  - Dependencies with `yield` (cleanup pattern)
- [x] **15. Middleware**
  - Built-in middleware
  - Custom middleware
  - CORS, GZip, TrustedHost
- [x] **16. Routers (APIRouter)**
  - Splitting code into multiple files
  - Tags, prefixes
  - Nested routers
- [x] **17. Form Data & File Uploads**

- `Form()`
- `UploadFile` vs `File()`
- Multiple files

- [x] **18. Headers & Cookies**
  - Reading them
  - Setting cookies in responses
- [x] **19. Static Files & Templates (Jinja2)**

🔵 PHASE 3 — DATABASE INTEGRATION

- [x] **20. Database Concepts Refresher**
  - SQL vs NoSQL
  - Connection pooling
  - ORM concept
- [x] **21. SQLAlchemy 2.0 (Sync)**
  - Engine, Session, Base
  - Models (table definitions)
  - Relationships (One-to-Many, Many-to-Many)
- [x] **22. SQLAlchemy with FastAPI**
  - DB session as dependency
  - CRUD operations (Create / Read / Update / Delete)
- [x] **23. Pydantic + SQLAlchemy together**
  - ORM mode / `from_attributes=True`
  - Schema vs Model separation
- [x] **24. Alembic (Database Migrations)**
  - Init, autogenerate, upgrade, downgrade
- [x] **25. Async SQLAlchemy**
  - `AsyncSession`, `async_engine`
  - When to use sync vs async DB
- [x] **26. SQLModel (alternative)**
  - Created by FastAPI's author
  - Pydantic + SQLAlchemy combined
- [x] **27. NoSQL Integration (Optional)**
  - MongoDB with Motor / Beanie
  - Redis for caching

PHASE 4 — ADVANCED FEATURES

- [x] **28. Async vs Sync Deep Dive**
  - `async def` vs `def`
  - Event loop basics
  - When NOT to use async
  - `run_in_threadpool`
- [x] **29. Authentication & Authorization**
  - Password hashing (`passlib`, `bcrypt`)
  - OAuth2 with Password Flow
  - JWT tokens (creation, validation, refresh)
  - `Depends(get_current_user)` pattern
  - Role-based access control (RBAC)
  - API keys
- [x] **30. Background Tasks**
  - `BackgroundTasks` (built-in)
  - When to use Celery / RQ / ARQ instead
- [x] **31. WebSockets**
  - Real-time communication
  - Connection manager pattern
  - Broadcasting
- [x] **32. Server-Sent Events (SSE) / Streaming Responses**
  - `StreamingResponse`
  - LLM streaming use case
- [x] **33. CORS (Cross-Origin Resource Sharing)**
  - Why it exists
  - Configuration for frontend integration
- [x] **34. Rate Limiting**
  - `slowapi` library
  - Per-IP, per-user
- [x] **35. Caching**
  - In-memory caching
  - Redis caching
  - `fastapi-cache2`
- [x] **36. Pagination**
  - Limit/offset
  - Cursor-based pagination

- [x] **37. Filtering, Sorting, Searching**
- [x] **38. Internationalization (i18n)** *(optional)*
- [x] **39. GraphQL with FastAPI** *(optional, via Strawberry)*

🔴 PHASE 5 — TESTING & QUALITY

- [x] **40. Testing Fundamentals**
  - `TestClient` (sync)
  - `httpx.AsyncClient` (async)
  - `pytest` basics
- [x] **41. Testing Endpoints**
  - GET, POST, with auth
  - Mocking dependencies (`app.dependency_overrides`)
- [x] **42. Database Testing**
  - Test database setup
  - Fixtures, transactions, rollbacks
- [x] **43. Coverage & CI**
  - `pytest-cov`
  - GitHub Actions basics

🟠 PHASE 6 — PRODUCTION & DEPLOYMENT

- [x] **44. Project Structure (Production)** `app/ api/ # routers core/ # config, security models/ # SQLAlchemy models schemas/ # Pydantic models services/ # business logic db/ # database setup tests/ main.py`
- [x] **45. Configuration Management**
  - `pydantic-settings`
  - `.env` files
  - Multiple environments (dev/staging/prod)
- [x] **46. Logging**
  - Structured logging (`structlog`, `loguru`)
  - Request ID tracing
- [x] **47. Security Best Practices**
  - HTTPS only
  - Secure headers
  - Input sanitization
  - SQL injection prevention

- Secret management

- [x] **48. Performance Optimization**
  - Async I/O usage
  - Connection pooling
  - Profiling (`py-spy`)
  - N+1 query problem
- [x] **49. Docker**
  - Dockerfile for FastAPI
  - `docker-compose` (app + DB + Redis)
  - Multi-stage builds
- [x] **50. Production Server**
  - Uvicorn vs Gunicorn vs Uvicorn workers
  - `gunicorn -k uvicorn.workers.UvicornWorker`
  - Reverse proxy (Nginx)
- [x] **51. Deployment Options**
  - VPS (Linux server basics)
  - Render / Railway / Fly.io (easy)
  - AWS (EC2, ECS, Lambda)
  - GCP Cloud Run
  - Kubernetes basics (optional)
- [x] **52. CI/CD Pipeline**
  - Auto-test → build → deploy
  - GitHub Actions example
- [x] **53. Monitoring & Observability**
  - Health check endpoints
  - Prometheus + Grafana
  - Sentry for error tracking
  - OpenTelemetry tracing
- [x] **54. API Versioning**
  - `/api/v1`, `/api/v2`
- [x] **55. API Documentation Customization**
  - Custom OpenAPI schema
  - Hiding endpoints
  - Adding examples

🎯 BONUS / EXPERT TOPICS

- [x] **56. Microservices Architecture with FastAPI**
- [x] **57. Event-Driven Systems**
  - RabbitMQ / Kafka integration
- [x] **58. gRPC alongside FastAPI**
- [x] **59. LLM/AI API Patterns**
  - Streaming responses
  - Token-by-token output
  - Rate limiting per user/token
- [x] **60. Final Capstone Project**
  - Full production-grade API combining everything
  - Orbit — a multi-tenant, real-time, AI-assisted work-management SaaS backend

---

📊 Course Stats

- **Total topics:** ~60
- **Estimated time:** 6–10 weeks (1–2 hrs/day)
- **Outcome:** Build, test, and deploy production-grade FastAPI apps confidently.

📁 Folder Convention

Each lesson lives in its own folder inside `Claude/`:

```
Claude/
├── SYLLABUS.md                        ← this file
├── Lesson 1 - What is FastAPI/
│   ├── theory.md
│   └── main.py
├── Lesson 2 - Installation & Setup/
│   ├── theory.md
│   └── main.py
└── ...
```

- `theory.md` → **Why / What / How** + real-world use case + mini task
- `main.py` → clean, runnable code for the topic